C++ Vector 完整的增删操作详解

📌 C++ vector 增删元素完整写法

vector 是 C++ 标准库的动态数组，增删操作分为**末尾操作**和**指定位置操作**，下面按场景分类说明：

一、添加元素（增）

1. 向末尾添加元素（最高效，O(1) 均摊复杂度）

`push_back()`

- 作用：在尾部**拷贝/移动**一个已存在的元素。
- 语法：`vec.push_back(元素值)`
- 示例：

Code block

```
vector<int> vec;
vec.push_back(10);  // 尾部添加 10
vec.push_back(20);  // 尾部添加 20 → vec: {10, 20}
```

`emplace_back()` （C++11 起推荐）

- 作用：在尾部**直接构造**元素，避免临时对象拷贝，效率更高（尤其自定义类型）。
- 语法：`vec.emplace_back(构造参数...)`
- 示例：

Code block

```
vector<string> vec;
vec.emplace_back("hello");  // 直接构造 string 对象，无需临时变量
// 等价于 vec.push_back(string("hello"));
```

2. 向指定位置插入元素（O(n) 复杂度，需移动后续元素）

`insert()`

- 作用：在迭代器指向位置插入元素（可插入单个、多个或一段范围）。
- 常用语法：

Code block

```
vector<int> vec = {1, 3};

// 1. 插入单个元素
vec.insert(vec.begin() + 1, 2);  // 在索引1位置插入2 → vec: {1, 2, 3}

// 2. 插入n个相同元素
vec.insert(vec.end(), 2, 0);     // 尾部插入2个0 → vec: {1, 2, 3, 0, 0}

// 3. 插入另一个容器的片段
vector<int> temp = {4, 5};
vec.insert(vec.end(), temp.begin(), temp.end());  // 尾部插入temp → vec: {1,2,3,0,0,4,5}
```

`emplace()` （C++11 起）

- 作用：在指定位置**直接构造**元素，避免拷贝，功能类似 `emplace_back()` 。
- 语法： `vec.emplace(迭代器, 构造参数...)`
- 示例：

Code block

```
vector<int> vec = {1, 3};
vec.emplace(vec.begin() + 1, 2);  // 在索引1位置构造2 → vec: {1, 2, 3}
```

二、删除元素（删）

1. 删除末尾元素（最高效，O(1) 复杂度）

`pop_back()`

- 作用：删除最后一个元素，无返回值。
- 语法： `vec.pop_back()`
- 示例：

Code block

```
vector<int> vec = {1, 2, 3};
vec.pop_back();  // 删除3 → vec: {1, 2}
```

2. 删除指定位置元素（O(n) 复杂度，需移动后续元素）

`erase()`

- 作用：删除迭代器指向位置的元素，或一段范围的元素。
- 常用语法：

Code block

```
vector<int> vec = {1, 2, 3, 4, 5};

// 1. 删除单个元素
vec.erase(vec.begin() + 2);  // 删除索引2的元素3 → vec: {1, 2, 4, 5}

// 2. 删除一段范围 [first, last)
vec.erase(vec.begin() + 1, vec.end());  // 删除索引1到末尾 → vec: {1}
```

3. 删除所有等于某值的元素（高效写法）

`erase()` + `std::remove()` （算法库）

- 作用：批量删除所有等于 `val` 的元素，是 LeetCode 「移除元素」题的标准解法。
- 语法：`vec.erase(remove(vec.begin(), vec.end(), val), vec.end())`
- 示例：

Code block

```
#include <algorithm>  // 必须包含algorithm头文件

vector<int> vec = {3, 2, 2, 3};
int val = 3;
// 删除所有等于3的元素 → vec: {2, 2}
vec.erase(remove(vec.begin(), vec.end(), val), vec.end());
```

4. 清空所有元素

`clear()`

- 作用：清空所有元素，`size()` 变为 0，但**不会释放内存**（`capacity()` 不变）。
- 语法：`vec.clear()`
- 示例：

Code block

```
vector<int> vec = {1, 2, 3};
vec.clear();  // vec 变为空，size=0，capacity仍为3
```

⚠ 关键注意事项

1. **迭代器失效**：
   - 插入/删除元素后，若触发 vector 扩容（`size() > capacity()`），所有旧迭代器、指针、引用都会失效；
   - 未扩容时，仅**插入/删除位置之后**的迭代器失效。
2. **性能差异**：
   - 末尾增删（`push_back` / `emplace_back` / `pop_back`）是 O(1) 均摊复杂度；
   - 中间增删（`insert` / `erase`）是 O(n) 复杂度（需移动后续元素），应尽量避免频繁使用。
3. **空间优化**：
   - 若想清空后释放内存，可在 `clear()` 后调用 `vec.shrink_to_fit()`，将 `capacity()` 收缩为当前 `size()`。

💡 推荐用法

- **末尾增删**：优先用 `emplace_back()` / `pop_back()`，高效简洁。
- **批量删值**：用 `erase(remove(...), vec.end())`，比遍历手动删除更高效。
- **原地修改**：若只需保留部分元素（如移除指定值），用**双指针法**原地覆盖，空间复杂度 O(1)（比如你之前做的 LeetCode 题）。

要不要我帮你写一个**完整的增删操作示例代码**，包含所有方法的运行效果，方便你直接测试？

（注：文档部分内容可能由 AI 生成）